# Álgebra Lineal Aplicada al Análisis de Datos

## Vectores, Matrices y el Modelo de Leontief

Juan Ignacio da Torre

Laboratorio de Métodos Cuantitativos Aplicados a la Gestión
Universidad de Buenos Aires

Primer cuatrimestre 2026

**Objetivo:** Comprender cómo el álgebra lineal transforma datos en decisiones económicas

# Objetivo de la charla

**¿Qué vamos a ver hoy?**

- Entender vectores y matrices como representación de datos económicos.
- Dominar operaciones matriciales fundamentales.
- Aplicar transformaciones matriciales al análisis de datos.
- Construir e interpretar el modelo insumo-producto de Leontief.
- Conectar conceptos matemáticos con código Python.

# ¿Por qué Álgebra Lineal?

Del dato aislado a las relaciones estructurales

# El álgebra lineal en la economía de los datos

**Datos económicos son multidimensionales:**

- Ventas de múltiples productos
- Precios en distintos mercados
- Flujos entre sectores económicos
- Transacciones inter-sectoriales

**El poder de las matrices**

Las matrices nos permiten:

- Representar relaciones complejas
- Operar sobre múltiples variables simultáneamente
- Modelar interdependencias económicas

*"Una matriz es más que números: es la estructura de las relaciones"*

# Vectores y Matrices

Los bloques fundamentales

# Matrices: Tablas de números con estructura

## Definición

Una matriz es un arreglo bidimensional de números organizados en filas y columnas.

**Ejemplo: Ventas por producto y región**

$$A = \begin{pmatrix} 100 & 150 & 200 \\ 80 & 90 & 110 \\ 120 & 130 & 140 \end{pmatrix}$$

- Filas: Productos (1, 2, 3)
- Columnas: Regiones (Norte, Centro, Sur)
- Elemento $a_{ij}$: Ventas del producto $i$ en región $j$

## En Python

```
import numpy as np
A = np.array([[100, 150, 200], [80, 90, 110], [120, 130, 140]])
```

# Vectores: Una lista ordenada de números

**Definición**

Un vector es una colección ordenada de números que representa magnitudes relacionadas.

**Ejemplos económicos:**

**Precios de productos:**

$$\vec{p} = \begin{pmatrix} 100 \\ 250 \\ 80 \end{pmatrix}$$

**Cantidades vendidas:**

$$\vec{q} = \begin{pmatrix} 50 \\ 30 \\ 120 \end{pmatrix}$$

¡Tu turno! — definir en Python

```
import numpy as np
p = # completar en clase
```

# Dimensiones y notación

**Dimensión de una matriz:**

- Una matriz de $m \times n$ tiene:
  - $m$ filas
  - $n$ columnas
- Elemento en posición $(i, j)$: $a_{ij}$

**Tipos especiales:**

- **Matriz cuadrada:** $m = n$
- **Matriz identidad:** $I_n$

$$I_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

*La dimensión determina qué operaciones son posibles*

Actividad en pizarrón

Dado un ejemplo numérico, identificar filas, columnas y elementos $a_{ij}$.

# Operaciones Matriciales

Transformando datos estructurados

# Suma y resta de matrices

**Regla**

Solo se pueden sumar/restar matrices de la misma dimensión, elemento por elemento.

**Ejemplo: Ventas de dos trimestres**

$$Q_1 = \begin{pmatrix} 100 & 150 \\ 80 & 90 \end{pmatrix}, \quad Q_2 = \begin{pmatrix} 120 & 160 \\ 85 & 95 \end{pmatrix}$$

¡Resolución en conjunto!

$$Q_1 + Q_2 = ?$$

En Python

```
Q1 = np.array([[100, 150], [80, 90]])
Q2 = np.array([[120, 160], [85, 95]])
# Completar en clase
```

# Multiplicación por escalar

## Regla

Multiplicar cada elemento de la matriz por un número (escalar).

**Ejemplo: Aumento del 10% en precios**

$$P = \begin{pmatrix} 100 \\ 250 \\ 80 \end{pmatrix}, \quad 1.10 \times P = \begin{pmatrix} 110 \\ 275 \\ 88 \end{pmatrix}$$

## En Python

```
P = np.array([[100], [250], [80]])
P_nuevo = 1.10 * P
print(P_nuevo)
```

*Útil para escenarios de crecimiento uniforme o ajustes*

# Multiplicación de matrices

## Regla fundamental

Para multiplicar $A_{m\times n}$ por $B_{n\times p}$:

- El número de columnas de $A$ debe ser igual al número de filas de $B$
- El resultado es una matriz $C_{m\times p}$

**Cálculo:**

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}$$

**Ejemplo simple:**

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \times \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

## Atención

$A \times B \neq B \times A$ (no es conmutativa)

# Multiplicación matricial en Python

**El operador @ es el que usamos para multiplicar matrices en Python**

En NumPy, `A @ B` realiza la multiplicación matricial estándar.

## Operador @ — ¡Resolución en conjunto!

```
import numpy as np

# Definir A y B (completar en clase)
A =
B =

# Multiplicacion matricial
C = A @ B
print(C)
```

# Transpuesta de una matriz

## Definición

La transpuesta $A^T$ se obtiene intercambiando filas por columnas.

**Ejemplo:**

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

## En Python

```
A = np.array([[1, 2, 3], [4, 5, 6]])
A_T = A.T
print(A_T)
```

*Útil para cambiar la estructura de los datos (filas ↔ columnas)*

# Inversa de una matriz

## Definición

La inversa $A^{-1}$ de una matriz cuadrada $A$ cumple: $A \times A^{-1} = A^{-1} \times A = I$

**Condición:** solo matrices cuadradas *no singulares* tienen inversa.

## En Python

```
A = np.array([[1, 4], [0, 6]])
A_inv = np.linalg.inv(A)
print(A_inv)
print(A @ A_inv)  # Deberia dar identidad
```

## Spoiler: la vamos a usar muy pronto

La inversa es la herramienta central del **Modelo de Leontief**: $(I - A)^{-1}$

# Transformación de Datos

Las matrices como operadores

# Matrices como transformaciones

**Concepto clave**

Multiplicar un vector por una matriz lo **transforma** en otro vector.

**Ejemplo económico: De cantidades a ingresos**

$$\underbrace{\begin{pmatrix} 100 & 0 & 0 \\ 0 & 250 & 0 \\ 0 & 0 & 80 \end{pmatrix}}_{\text{Matriz de precios (diagonal)}} \times \underbrace{\begin{pmatrix} 50 \\ 30 \\ 120 \end{pmatrix}}_{\text{Cantidades}} = \underbrace{\begin{pmatrix} 5000 \\ 7500 \\ 9600 \end{pmatrix}}_{\text{Ingresos}}$$

*La matriz de precios transforma cantidades en ingresos*

# Agregación con matrices

**Problema:** Calcular ventas totales por región sumando todos los productos.

$$\underbrace{\begin{pmatrix} 100 & 150 & 200 \\ 80 & 90 & 110 \\ 120 & 130 & 140 \end{pmatrix}}_{\text{Ventas por producto y región}} \xrightarrow{\text{sumar filas}} \underbrace{\begin{pmatrix} 450 \\ 280 \\ 390 \end{pmatrix}}_{\text{Total por región}}$$

**Con multiplicación matricial:**

$$\underbrace{\begin{pmatrix} 1 & 1 & 1 \end{pmatrix}}_{\text{Vector de unos}} \times \begin{pmatrix} 100 & 150 & 200 \\ 80 & 90 & 110 \\ 120 & 130 & 140 \end{pmatrix} = \begin{pmatrix} 300 & 370 & 450 \end{pmatrix}$$

*La elección del vector determina qué dimensión se agrega*

# Cambio de unidades/base

**Ejemplo:** Convertir ventas en 3 monedas a una moneda base (USD).

**Tasas de cambio (vector columna):**

- 1 EUR = 1.10 USD
- 1 GBP = 1.30 USD
- 1 JPY = 0.0075 USD

$$T = \begin{pmatrix} 1.10 \\ 1.30 \\ 0.0075 \end{pmatrix}$$

**Ventas (vector columna):**

$$\vec{v} = \begin{pmatrix} 1000 \text{ EUR} \\ 800 \text{ GBP} \\ 50000 \text{ JPY} \end{pmatrix}$$

**Operación matricial — multiplicamos $T^T$ por el vector de ventas:**

$$T^T \cdot \vec{v} = \begin{pmatrix} 1.10 & 1.30 & 0.0075 \end{pmatrix} \cdot \begin{pmatrix} 1000 \\ 800 \\ 50000 \end{pmatrix} = 1100 + 1040 + 375 = \mathbf{2515} \text{ USD}$$

*Las matrices permiten cambios de representación sistemáticos*

# Modelo de Leontief

Análisis de interdependencia sectorial

# El modelo insumo-producto de Leontief

**¿Qué es?**

Un modelo que describe cómo los sectores de una economía dependen unos de otros para producir.

**Idea central:**

- Cada sector produce un bien
- Para producir necesita insumos de otros sectores
- La producción de un sector afecta a los demás

**Premio Nobel (1973):**

- Wassily Leontief
- Por su trabajo en análisis input-output
- Aplicable a economías reales

*"Para saber cuánto producir, necesito saber cuánto necesitan los demás"*

# Tabla insumo-producto (Input-Output)

**Ejemplo simple con 3 sectores:**

| | **Consumo intermedio** | | | **Demanda Final** | **Producto Total** |
|---|---|---|---|---|---|
| | Sector 1 | Sector 2 | Sector 3 | | |
| Sector 1 | 50 | 100 | 50 | 1000 | 1200 |
| Sector 2 | 100 | 100 | 100 | 1000 | 1300 |
| Sector 3 | 100 | 300 | 100 | 1500 | 2000 |

- **Consumo intermedio:** Cuánto usa cada sector de los demás
- **Demanda final:** Consumo de hogares, gobierno, exportaciones
- **Producto total:** Consumo intermedio + Demanda final

# Matriz de coeficientes técnicos

**Definición**

$a_{ij}$ = Cantidad del sector $i$ necesaria para producir 1 unidad del sector $j$

$$a_{ij} = \frac{\text{Insumo del sector } i \text{ al sector } j}{\text{Producto total del sector } j}$$

**Ejemplo — cuánto necesita cada sector de insumo para esa cantidad de productos:**

$$A = \begin{pmatrix} 50/1200 & 100/1300 & 50/2000 \\ 100/1200 & 100/1300 & 100/2000 \\ 100/1200 & 300/1300 & 100/2000 \end{pmatrix} \approx \begin{pmatrix} 0.042 & 0.077 & 0.025 \\ 0.083 & 0.077 & 0.050 \\ 0.083 & 0.231 & 0.050 \end{pmatrix}$$

*Cada columna representa la "receta" de producción de un sector*

# Ecuación fundamental de Leontief

**Relación básica:**

$$X = AX + H$$

Donde:

- $X$: Vector de producto total de cada sector
- $A$: Matriz de coeficientes técnicos
- $H$: Vector de demanda final
- $AX$: Consumo intermedio total

**Despejando:**

$$X - AX = H$$
$$(I - A)X = H$$
$$X = (I - A)^{-1}H$$

Interpretación

$(I - A)^{-1}$ es la **matriz de multiplicadores de Leontief**: muestra el impacto total en toda la economía de un cambio en la demanda final.

# Interpretación económica

## La matriz $(I - A)^{-1}$

Cada elemento $l_{ij}$ indica cuánto debe producir el sector $i$ en total para satisfacer 1 unidad adicional de demanda final del sector $j$.

**Incluye efectos directos e indirectos:**

1. **Efecto directo:** Producción necesaria del sector demandado
2. **Efecto indirecto:** Producción adicional de sectores proveedores
3. **Efecto inducido:** Producción de proveedores de proveedores, etc.

## Ejemplo

Si aumenta la demanda de autos en 100 unidades, $(I - A)^{-1}$ nos dice cuánto debe aumentar la producción de acero, plástico, electrónica, etc.

# Análisis de impacto con Leontief

**Preguntas que podemos responder:**

1. ¿Cuánto debe producir cada sector si la demanda de uno cambia?
2. ¿Qué sectores son más interdependientes?
3. ¿Cuál es el efecto multiplicador de una inversión sectorial?
4. ¿Cómo afecta un shock externo a toda la economía?

## Aplicaciones reales

- Planificación económica nacional
- Análisis de cadenas de suministro
- Evaluación de políticas industriales
- Estudio de impacto ambiental (matrices extendidas)

## Clave para TPs y parciales

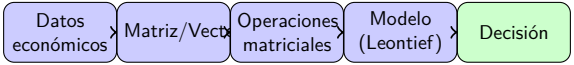
Esta noción es fundamental para justificar respuestas: ante un cambio en la demanda final, ¿cuánto varía la producción de cada sector? Pensar este razonamiento tanto para el TP como para el parcial a papel.

# Conectando Todo

De la teoría a la práctica

$\Rightarrow$ **A partir de aquí: pasamos a Google Colab**

# Flujo de trabajo completo




**Herramientas en cada etapa:**

- **Datos:** Pandas (DataFrames), archivos CSV/Excel
- **Matrices:** NumPy arrays
- **Operaciones:** NumPy (`@`, `.T`, `np.linalg.inv`)
- **Modelo:** Álgebra lineal + interpretación económica
- **Decisión:** Visualización + análisis de resultados

# De DataFrames a matrices: Puente conceptual

## Pandas → NumPy

```
import pandas as pd
import numpy as np

# Leer datos de tabla input-output
df = pd.read_csv('tabla_io.csv', index_col=0)

# Convertir a matriz NumPy
matriz_io = df.values  # o df.to_numpy()

# Ahora podemos aplicar algebra lineal
A = matriz_io / producto_total  # Coeficientes tecnicos
```

*Pandas organiza los datos, NumPy los opera matemáticamente*

# Buenas prácticas en análisis matricial

1. **Verificar dimensiones:** Antes de multiplicar, revisar compatibilidad
2. **Usar nombres descriptivos:** `coef_tecnicos`, no solo `A`
3. **Documentar supuestos:** Qué representa cada fila/columna
4. **Validar resultados:** ¿Tienen sentido económico?
5. **Chequear singularidad:** Antes de invertir, verificar que la matriz no sea singular

## Errores comunes

- Confundir `*` con `@` en Python
- No verificar dimensiones antes de operar
- Invertir matrices mal condicionadas
- Perder el significado económico en los cálculos

## Tip: estos criterios son evaluables en el parcial

Saber identificar y evitar estos errores es parte de la competencia esperada.

# Extensiones del modelo

**El álgebra lineal permite ir más allá:**

- **Modelos dinámicos:** Incorporar tiempo (matrices de transición)
- **Análisis ambiental:** Matriz extendida con emisiones/contaminación
- **Cadenas de valor globales:** Leontief multi-país
- **Análisis estructural:** Identificar sectores clave
- **Optimización:** Combinado con programación lineal

**El concepto permanece**

Las matrices como herramienta para modelar interdependencias complejas.

# Cierre

$$\text{Datos} \xrightarrow{\text{Vectores/Matrices}} \text{Relaciones} \xrightarrow{\text{Operaciones}} \text{Insights} \xrightarrow{\text{Modelos}} \text{Decisiones}$$

**Lo que aprendimos hoy:**

- Las matrices representan relaciones estructuradas entre datos
- Las operaciones matriciales transforman y agregan información
- El modelo de Leontief captura interdependencias económicas
- Python/NumPy traduce estos conceptos a código ejecutable

*El álgebra lineal no es solo matemática: es el lenguaje de las relaciones económicas*

# Trabajo Práctico 1

**El TP1 integra todas las actividades con una base de datos real:**

1. Explorar una tabla insumo-producto real (p.ej. datos del INDEC u otra fuente)
2. Convertirla en un Pandas DataFrame
3. Calcular los coeficientes técnicos
4. Incorporar una fila que muestre el valor agregado por sector
5. Suponer un cambio en la demanda final (+30% en un sector)
6. Calcular cómo varía el producto total usando Leontief
7. Interpretar los resultados económicamente

**Formato:** Notebook de Google Colab documentado

**Importante**

El TP1 mezcla todas las actividades propuestas con una base de datos real. No es un ejercicio hipotético: van a trabajar con datos concretos de una economía real.

## Preguntas

# ¿Preguntas?

*"Una buena pregunta sobre matrices vale más que mil operaciones sin sentido"*

# Fuentes y bibliografía


- Fraquelli, A. D. & Gache, A. L. (2019). *Notas de álgebra teórico-prácticas*. FCE-UBA.
- Hilpisch, Y. (2018). *Python for Finance: Mastering Data-Driven Finance* (2nd ed.). O'Reilly Media.
- Miller, R. E. & Blair, P. D. (2009). *Input-Output Analysis: Foundations and Extensions*. Cambridge University Press.
- NumPy Documentation: `https://numpy.org/doc/`
- Leontief, W. (1986). *Input-Output Economics* (2nd ed.). Oxford University Press.


Materiales educativos con fines académicos.